

Expression Evaluation Architectures

Technical Comparison: Stack-based Postfix vs. Expression Trees

Overview: When parsing and calculating mathematical or logical statements in computer science, choosing the underlying structure dictates memory consumption, structural flexibility, and evaluation paradigms. Below is a direct structural and functional comparison between linear stack-based postfix notation and hierarchical expression trees.

FEATURE	STACK-BASED POSTFIX	EXPRESSION TREE
Data Structure	Stack	Binary Tree
Evaluation	Direct	Recursive traversal
Time Complexity	O(n)	O(n)
Space Complexity	O(n)	O(n)
Stores Expression Structure	No	Yes
Preorder Available	No	Yes
Inorder Available	No	Yes
Postorder Available	Input itself	Yes
Easy Structural Modification	No	Yes
Main Purpose	Direct evaluation	Representation and evaluation

Stack-based Postfix Summary

Optimized for straightforward, linear processing environments. It eliminates operator precedence parsing overhead during evaluation, making it ideal for interpreters and lightweight virtual machines.

Expression Tree Summary

Provides a complete hierarchical abstract syntax representation. Essential for compilers, symbolic differentiation, code optimization passes, and transformations where the structure must be manipulated.